

DigitalPulse

統合型心電図解析・患者管理システム

要件定義書（コンセプトシート）

文書番号	DP-RD-001
版数	1.0
作成日	2026年5月5日
作成者	辰田
配布対象	開発チーム・面接官・レビュアー

本書は学術・ポートフォリオ目的で作成されています

© 2026 DigitalPulse Medical System

改訂履歴

版数	日付	作成／改訂者	改訂内容
1.0	2026年5月5日	辰田	初版作成

目次

改訂履歴	2
目次	3
1. エグゼクティブサマリ	4
2. 開発背景 (Why We Built This)	5
2.1 解決したい社会課題	5
2.2 なぜ今このシステムを作るのか	5
2.3 開発者の原体験・動機	5
3. ターゲットユーザーと課題整理	6
3.1 ペルソナ定義	6
3.2 課題と解決策のマッピング (Problem-Solution Fit)	6
4. システムスコープ	7
4.1 スコープ内 (In-Scope)	7
4.2 スコープ外 (Out-of-Scope)	7
4.3 システム境界	7
5. 主要機能一覧	9
5.1 コア機能 (AI 解析・心電図)	9
5.2 管理機能 (患者・ダッシュボード)	9
5.3 出力機能 (レポート)	10
5.4 基盤機能 (認証・セキュリティ)	10
6. 非機能要件 (概要)	11
7. 技術スタック選定理由	12
7.1 フロントエンド: Next.js 15 (App Router)	12
7.2 バックエンド: Spring Boot 3.4 (Java 21)	12
7.3 AI サービス: FastAPI + TensorFlow 2.15	12
7.4 AI モデル: 1D CNN (自作)	13
8. 開発上の工夫・技術的チャレンジ	14
8.1 セキュア・レイアウト・パターンの実装	14
8.2 クロスドメイン認証と CORS の克服	14
8.3 無停止スキーマ移行	14
9. KPI・成功指標	15
9.1 ポートフォリオとしての評価指標	15
9.2 医療現場適用時の効果試算 (参考)	15
10. 今後のロードマップ	16
11. 用語集	17
12. 承認欄	18

1. エグゼクティブサマリ

DigitalPulse は、「心電図（ECG）の自動解析」と「患者情報の統合管理」を一つのプラットフォームで実現する、医療従事者向けの次世代 Web アプリケーションである。

【ひとことで言うと】

医師が心電図を目視判定する負担を AI で削減し、
「見逃し」と「記録漏れ」をゼロに近づける診療支援プラットフォーム。

現在の医療現場では、1 日あたり数百件に上る心電図をすべて医師が目視で確認しなければならない。判定ミスや記録漏れが生じるリスクは常に存在し、特に夜間・休日の当直帯ではその影響が顕著になる。本システムは MIT-BIH 不整脈データベースで訓練した CNN モデルと OpenAI GPT-4o-mini を組み合わせることで、不整脈の自動分類・所見文章の自動生成・電子カルテ記載ワークフローの効率化を同時に実現する。

2. 開発背景（Why We Built This）

2.1 解決したい社会課題

日本の医療機関では深刻な医師不足・過重労働が社会問題となっている。循環器系疾患は死因の第2位を占め（2022年厚生労働省統計）、心電図検査は最も基本的かつ頻度の高い検査の一つである。にもかかわらず、以下の課題が長年放置されてきた。

#	課題	具体的な問題
1	読影業務の属人化	熟練医師の経験と勘に依存。医師交代時に判定基準がブレる
2	記録・転記の工数	解析結果を電子カルテへ手動入力する二重作業。入力ミスのリスクもある
3	見逃しリスク	疲労状態での読影では不整脈の見逃しが発生する。特に夜間帯で深刻
4	患者情報の分散管理	検査履歴・波形データ・医師所見が別々のシステムに散在し、統合参照が困難

2.2 なぜ今このシステムを作るのか

AI・クラウド技術の急速な普及により、かつては専門ベンダーにしかできなかった医療 AI の開発が、個人開発者レベルでも現実的なコストと精度で実現可能になった。特に以下の3点が本プロジェクトを後押しした。

- ▶ TensorFlow / Keras の成熟と MIT-BIH データセットの公開により、不整脈分類 AI を自力で構築・評価できる環境が整った
- ▶ OpenAI API (gpt-4o-mini) の登場により、高品質な医療文書の自動生成が API 呼び出し 1 回で実現できるようになった
- ▶ Vercel・Render・Google Cloud Run といった PaaS の普及により、本番相当のマルチクラウド環境を無料または低コストで構築・公開できるようになった

上記の技術的追い風を受け、「学習目的であっても、現場の課題を正面から捉えたシステムを作る」という方針のもと本プロジェクトを立ち上げた。

2.3 開発者の原体験・動機

開発者（辰田）は、家族が循環器疾患で入院した経験を持つ。入院中、心電図検査の結果を医師が口頭で伝えるだけで、患者・家族が波形の意味を理解できないことへの違和感を覚えた。また、看護師が手書きで記録を転記する姿も印象的だった。「AI と Web 技術を組み合わせれば、この非効率を解消できる」という直感が、本システムの着想に繋がっている。

3. ターゲットユーザーと課題整理

3.1 ペルソナ定義

本システムの主要ターゲットを以下の 3 ペルソナに設定する。

属性	ペルソナ A（主）	ペルソナ B（副）	ペルソナ C（将来）
職種	循環器科 / 内科医師	看護師 / 検査技師	患者 / 家族
環境	200 床規模の中規模病院	同上 / クリニック	在宅 / 定期検診
主な悩み	1 日 200 件超の心電図読影。当直時の疲労による判定ブレ	手書き記録 → カルテ転記の繰り返し。漏れ・誤記が怖い	検査結果を平易な言葉で説明してほしい
得られる価値	AI 一次判定 + 所見案の自動生成。確認・修正のみで記録完了	CSV インポートで一括登録。転記作業が不要になる	（将来機能）波形と平易な解説を患者向けに表示

3.2 課題と解決策のマッピング（Problem-Solution Fit）

#	ユーザーの課題（Pain）	本システムの解決策（Gain）	対応機能 ID
P1	心電図の読影に時間がかかり、他の業務を圧迫する	CNN モデルが 187 点波形を即座に 5 分類し、一次判定を提示する	F-004（AI 自動判定）
P2	電子カルテへの所見入力 が定型文の繰り返しで非効率	GPT-4o-mini が判定結果をもとに 2~4 文の所見案を自動生成。医師は確認・修正のみ	F-005（AI 所見生成）
P3	大量データの取り込みが 手作業で、ミスが発生する	MIT-BIH 形式 CSV を一括インポートし、同時に AI 解析まで完結させる	F-006（CSV インポート）
P4	患者情報と検査履歴が別システムに分散し、参照に手間がかかる	患者 ⇄ 心電図レコードを紐づけて一元管理。患者 ID でのフィルタリングも可能	F-002（患者管理）
P5	臨床報告書の作成に時間がかかる	波形グラフ・AI 診断・医師所見を統合した A4 レポートをワンクリックで PDF 出力	F-007（PDF 出力）
P6	大量のデータ読み込みによるパフォーマンス低下	DT0Iによるデータ軽量化とページネーション	

4. システムスコープ

4.1 スコープ内（In-Scope）

本バージョン（v1.0）で対応する機能・範囲を以下に示す。

- ▶ 心電図波形データ（MIT-BIH 形式 187 点）のアップロード・管理
- ▶ CNN モデルによる 5 分類不整脈自動判定（Normal / SVEB / VEB / Fusion / Unknown）
- ▶ OpenAI GPT-4o-mini を活用した所見テキストの自動生成
- ▶ 患者情報（氏名・年齢・性別）の CRUD 管理
- ▶ 患者と心電図レコードの紐づけ・フィルタリング
- ▶ インタラクティブ波形チャートの表示（Recharts）
- ▶ 臨床レポートの PDF エクスポート（html2pdf.js）
- ▶ ユーザー認証・セッション管理（NextAuth.js）
- ▶ マルチクラウド本番環境へのデプロイ（Vercel / Render / Cloud Run / Neon）

4.2 スコープ外（Out-of-Scope）

以下は本バージョンの対象外とし、将来のバージョンアップで検討する。

- ▶ リアルタイム心電図モニタリング（ストリーミング入力）
- ▶ 医療機器認証・薬機法への対応（本システムは学術・ポートフォリオ目的）
- ▶ 患者向けポータル機能（検査結果の患者本人への開示）
- ▶ 他院の電子カルテシステム（HIS）との API 連携
- ▶ 多施設・マルチテナント対応
- ▶ モバイルネイティブアプリ（iOS / Android）

4.3 システム境界

区分	要素	本システムとの関係
システム内	Next.js フロントエンド	Vercel にホスティング。ユーザーが直接操作する UI
システム内	Spring Boot API	Render にホスティング。ビジネスロジックと DB 管理
システム内	FastAPI AI サービス	Cloud Run にホスティング。AI 推論エンジン
システム内	PostgreSQL (Neon)	患者情報・ECG レコードの永続化

区分	要素	本システムとの関係
外部サービス	OpenAI API (gpt-4o-mini)	AI サービスから呼び出す。所見テキスト生成に使用
外部データ	MIT-BIH 不整脈データベース	AI モデルの学習・検証に使用（本番では直接参照しない）
外部アクター	医師 / 検査技師（ユーザー）	ブラウザ経由でフロントエンドを操作する

5. 主要機能一覧

5.1 コア機能（AI 解析・心電図）

ID	機能名	説明	利用技術
F-003	心電図波形可視化	187 点の波形データをインタラクティブなエリアチャートで表示。末尾ゼロパディングを自動トリミングし、心拍を強調。異常時は赤、正常時は青のグラデーション表示	Recharts (AreaChart)
F-004	AI 自動判定	MIT-BIH で学習した 1D CNN が波形を 5 クラスに自動分類 (Normal / SVEB / VEB / Fusion / Unknown)。信頼度 (Confidence) もパーセント表示	TensorFlow 2.15 / Keras 1D CNN
F-005	AI 所見テキスト生成	判定結果と信頼度をプロンプトに組み込み、電子カルテ記載に直接使えるレベルの所見文・方針案を 2~4 文で自動生成。正常時・異常時でプロンプトを切り替え	OpenAI gpt-4o-mini (temperature=0.3)
F-006	CSV 一括インポート	MIT-BIH 形式 (187 列) CSV をアップロードするだけで全行を一括取り込み・AI 解析・DB 保存まで完結。患者 ID 指定で自動紐づけも可能	Apache Commons CSV / Spring Multipart

5.2 管理機能（患者・ダッシュボード）

ID	機能名	説明	利用技術
F-001	ダッシュボード	全解析件数・異常検知数・未確認レポート数をリアルタイムカード形式で表示。患者 ID と異常フラグによる動的フィルタリングで緊急データを即時抽出	Next.js App Router / React State
F-002	患者管理 (CRM)	患者の新規登録（氏名・年齢・性別）、一覧参照、個別詳細・検査履歴の時系列表示。患者 ID フィルタでダッシュボードとシームレスに連携	Spring Boot REST API / JPA / PostgreSQL
F-009	AI 一括再解析	所見未記録レコードを最大 50 件まとめて非同期でバックグラウンド解析。蓄積したデータへの一括 AI 適用がバッチ的に行える	Spring @Async / @Transactional

5.3 出力機能（レポート）

ID	機能名	説明	利用技術
F-007	PDF レポート出力	患者情報・AI 診断結果・波形グラフ・医師所見を A4 1 枚に統合した臨床レポートをワンクリックで PDF 出力。ファイル名は「ECG_Report_#ID.pdf」	html2pdf.js（クライアントサイド）
F-008	医師コメント編集・保存	AI 生成の所見を医師がそのまま使えるほか、内容を修正・追記して保存できる。変更後の内容は次回 PDF 出力に反映される	PUT /api/ecg/{id}/comment

5.4 基盤機能（認証・セキュリティ）

ID	機能名	説明	利用技術
F-010	認証・セッション管理	ユーザー名・パスワードによるログイン。AppLayout コンポーネントが未認証状態を検知してトップにリダイレクト。HttpOnly Cookie でセッションを安全に保持	NextAuth.js CredentialsProvider
F-011	クロスドメイン CORS 管理	Vercel（フロントエンド）と Render（API）の異なるドメイン間で HttpOnly Cookie を共有するため、allowedOriginPatterns + allowCredentials を厳密に設定	Spring Security WebConfig

6. 非機能要件（概要）

詳細は基本設計書（DP-BD-001）第 7 章を参照すること。本書では就活・ポートフォリオ評価に特に重要な観点をサマリで示す。

観点	目標値・方針	実現手段
可用性	フロントエンド 99.99%（Vercel SLA）	Vercel エッジネットワーク。複数リージョンに自動分散
応答速度	AI 推論含む総処理 5 秒以内	CNN 推論 < 100ms。OpenAI API 応答は非同期で体験を損なわない設計
セキュリティ	認証・CORS・環境変数の三重管理	HttpOnly Cookie / allowedOriginPatterns / Cloud Run シークレット環境変数
拡張性	マイクロサービス構成で個別スケーリング	Frontend / API / AI を独立コンテナで分離。AI だけ GPU インスタンスに移行可能
保守性	GitHub 連動による自動デプロイ	main ブランチ push → Vercel / Render が自動ビルド・デプロイ

7. 技術スタック選定理由

ここでは「なぜその技術を選んだのか」を採用担当者・技術評価者に説明できるよう、選定の根拠を記録する。

7.1 フロントエンド：Next.js 15 (App Router)

React のみでも実装可能であったが、以下の理由で Next.js を選択した。

- ▶ App Router による直感的なファイルシステムルーティングで開発速度が向上する
- ▶ NextAuth.js との公式統合により、認証実装のボイラープレートを大幅に削減できる
- ▶ Vercel との親和性が非常に高く、push 一発でデプロイ・プレビュー環境が整う
- ▶ Server Components / Client Components の分離により、将来的なパフォーマンス最適化の余地がある

7.2 バックエンド：Spring Boot 3.4 (Java 21)

Node.js (Express) や Python (FastAPI) で実装することも検討したが、以下の観点で Spring Boot を採用した。

- ▶ Spring Data JPA により、SQL を書かずにリポジトリパターンで DB 操作が実現できる
- ▶ Spring Security の組み合わせでエンタープライズレベルの認証・認可を標準実装できる
- ▶ Java 21 の Virtual Threads により、I/O バウンドな処理に有利
- ▶ 就活において「エンタープライズ Java の知見がある」ことをアピールできる

7.3 AI サービス：FastAPI + TensorFlow 2.15

AI 推論エンジンを Spring Boot とは別サービスに分離した理由は以下の通りである。

- ▶ TensorFlow は Python エコシステムの学習リソースが最も豊富
- ▶ AI サービスのみ Google Cloud Run に配置することで、将来的に GPU インスタンスへの移行が容易
- ▶ FastAPI の非同期 I/O (async/await) により、OpenAI API 応答待ち中も他リクエストを処理できる

7.4 AI モデル：1D CNN（自作）

既存の心電図 AI API を使う選択肢もあったが、以下の理由で自力実装を選択した。

- ▶ 「MIT-BIH データから自分でモデルを設計・学習させた」というポートフォリオの差別化になる
- ▶ モデルの入力形式（187 点）・出力形式（5 クラス softmax）を自分でコントロールでき、API 仕様に組み込みやすい
- ▶ TensorFlow / Keras の実務的な使い方を学べる機会として価値が高い

8. 開発上の工夫・技術的チャレンジ

本章は就活面接での「苦労した点・工夫した点」の回答を文書化したものである。

8.1 セキュア・レイアウト・パターンの実装

【課題】ログアウト後もサイドバーが残存し、直リンクで患者情報にアクセスできてしまう脆弱性

【解決策】AppLayout コンポーネントによる「番人ロジック」
→ useSession + usePathname を組み合わせた未認証リダイレクト + 条件付きサイドバー表示

Next.js のページコンポーネントは認証状態に関係なくレンダリングされる点に気づかず、当初はセッション切れ後もサイドバーが表示される状態になっていた。解決策として、すべてのページを AppLayout でラップし、NextAuth の status フラグを監視する「番人ロジック」を実装した。この設計は React のレンダリングライフサイクルへの深い理解が求められ、フロントエンド設計の学びになった。

8.2 クロスドメイン認証と CORS の克服

【課題】Vercel（フロント）と Render（API）という異なるドメイン間での HttpOnly Cookie の共有

【解決策】Spring Boot の allowedOriginPatterns + allowCredentials(true)
→ フロント側も credentials: 'include' を徹底することで解決

SameSite 属性と CORS ポリシーの相互作用は、最初はエラーメッセージの意味が全く分からなかった。ブラウザの DevTools でレスポンスヘッダを詳細に調べ、allowedOriginPatterns をワイルドカードではなく厳密なオリジン指定にすることが HttpOnly Cookie 共有の前提条件であることを理解した。ネットワーク・セキュリティの実践的な知識が深まった経験となった。

8.3 無停止スキーマ移行

【課題】外部キー制約が張られたカラムへの AUTO_INCREMENT 追加で Incompatible エラー

【解決策】FOREIGN_KEY_CHECKS の一時的無効化 + 制約再構築スクリプト
→ データの整合性を維持したままスキーマ最適化に成功

「本番 DB を止めずにスキーマ変更する」というシチュエーションは現場でも頻出する課題である。外部キー制約と AUTO_INCREMENT の相性問題を経験し、制約の一時解除 → ALTER → 再構築という手順を安全に実行するスクリプトを作成した。データベース運用の実践知識を得た貴重な経験となった。

9. KPI・成功指標

9.1 ポートフォリオとしての評価指標

#	指標	目標	達成状況（v1.0）
①	本番環境での公開（デプロイ完了）	URL アクセス可能	✓ 達成
②	AI 推論の動作確認（正常 / 異常の分類）	5 クラス分類が動作する	✓ 達成
③	AI 所見テキストの自動生成	GPT-4o-mini で所見文が出力される	✓ 達成
④	マルチクラウド構成のデプロイ	3 クラウド以上に分散構成	✓ 達成（4 サービス）
⑤	PDF レポートの出力	A4 PDF が生成できる	✓ 達成
⑥	設計書類の整備（要件定義 / 基本 / 詳細）	3 種類の文書を Word 形式で作成	✓ 達成（本書含む 3 点）

9.2 医療現場適用時の効果試算（参考）

以下は、仮に本システムを 200 床規模の病院で運用した場合の効果試算である（実証値ではなく、一般的な業務時間データに基づく仮定）。

業務	現状（手動）	本システム導入後	削減率（試算）
心電図 1 件の読影時間	平均 3 分（熟練医師）	AI 判定確認：30 秒	▲ 83%（試算）
所見文の作成時間	平均 5 分（定型文入力）	確認・修正：1 分以内	▲ 80%（試算）
CSV データ取り込み（100 件）	30～60 分（手作業）	約 2～3 分（一括処理）	▲ 95%（試算）

※ 本試算はポートフォリオ目的の参考値であり、実際の効果を保証するものではありません。

10. 今後のロードマップ

フェーズ	時期（目安）	主な開発内容	優先理由
v1.0 完了済み		ページネーション	
v1.1 セキュリティ強化	～1 ヶ月	パスワード bcrypt 化 / CSRF 対策 / AI サービスの CORS を特定オリジンに限定	既知の課題を解消し、セキュリティ品質を高める
v1.2 UX 改善	～3 ヶ月	検索インデックス最適化 / ダッシュボードのチャート強化	データ増加時のパフォーマンス劣化を防ぐ
v2.0 モデル精度向上	～6 ヶ月	モデルを LSTM / Transformer ベースに刷新。精度評価レポートの自動生成	AI 分類精度を高め、臨床利用可能な水準に近づける
v2.1 連携拡張	～12 ヶ月	HL7 FHIR 形式の電子カルテ API 連携 / 患者向けポータル	実医療環境への適合性を高める

本バージョンではデータの不変性を担保するため、GUIからの削除機能を非搭載としている。削除が必要な場合はDBAが対応する運用を想定する。将来的に、特権管理者（Admin）ロール向けに『論理削除機能』および『操作ログ記録機能』の実装を検討する。

11. 用語集

用語	説明
ECG / 心電図	Electrocardiogram。心臓の電氣的活動を記録した波形。P 波・QRS 波・T 波から構成される
MIT-BIH データベース	MIT とベス・イスラエル病院が公開する標準的な不整脈注釈データセット。心電図 AI モデルのベンチマークに広く使用される
SVEB	Supraventricular Ectopic Beat（上室性期外収縮）。心房・房室接合部から発生する異常拍
VEB	Ventricular Ectopic Beat（心室性期外収縮）。心室から発生する異常拍。致死性不整脈のリスクがある
Fusion Beat	心室融合不整脈。正常洞調律拍と心室性拍が融合した波形
CNN	Convolutional Neural Network（畳み込みニューラルネットワーク）。1D CNN で時系列の局所パターンを効率的に学習する
GPT-4o-mini	OpenAI の高速・低コストな LLM。本システムでは医療所見テキストの生成に使用する
HttpOnly Cookie	JavaScript から読み取れない Cookie 属性。XSS 攻撃でセッション情報が盗まれるリスクを低減する
CORS	Cross-Origin Resource Sharing。異なるオリジンからの HTTP リクエストを制御するブラウザのセキュリティ機能
PaaS	Platform as a Service。Vercel・Render・Neon・Google Cloud Run など、インフラ管理不要でアプリを公開できるクラウドサービス
HL7 FHIR	医療情報の標準交換規格。将来的な電子カルテシステム連携に必要

12. 承認欄

役割	氏名	署名	承認日
作成者	辰田		2026年5月5日
レビュー者			
承認者			

以上

本書は *DigitalPulse* のポートフォリオ用要件定義書です。医療行為・診断の根拠として使用することを意図したものではありません。